

What is Scope?

In Java, **Scope** defines the visibility and availability of a variable. Variables only exist within the specific code blocks where they are declared.

Method Scope

Variables declared inside a method are local to that method. They cannot be accessed by other methods in the same class.

```
public static void main(String[] args) {  
    int x = 100; // Accessible from this point onwards  
    System.out.println(x);  
}
```

Block Scope

Any code between curly braces { } is a block. Variables declared inside a block expire the moment the closing brace is reached.

```
{  
    int temp = 50;  
    System.out.println(temp); // temp works here  
}  
// Error: temp cannot be resolved to a variable
```

Loop & Conditional Scope

This is a specific type of block scope. Variables like loop counters (`i`) are destroyed once the loop completes.

```
for (int i = 0; i < 5; i++) {  
    // i exists here  
}  
// i is now out of scope
```

Common Implementation Mistake

Attempting to access a variable outside its defined block will trigger a compile-time error.

```
java: cannot find symbol  
symbol: variable myNumber
```

Why Scope Management Matters

- **Memory Efficiency:** Java cleans up variables no longer in scope.
- **Security:** Limits data access to only where it is required.
- **Name Reuse:** Allows the same variable names (like `i`) in separate loops.